

Building a Semantic Product Recommendation System

Practical Tutorial Using Sentence Transformers, ChromaDB, and Ollama

By : Surya Jade Edakunni

Independent Study - INSY 661

Instructor: Professor Animesh Animesh

Date: 28th August 2026

Overview

This tutorial explains how to build a semantic search system that retrieves information based on meaning rather than exact keywords. The process uses *Sentence Transformers* to create embeddings, *ChromaDB* for vector retrieval, and *Ollama* to add natural-language interpretation and response generation.

Step 1: Prepare Your Dataset

Start with a dataset containing descriptive information that users may want to search.

Decide what belongs in the embedding:

- Product/item name
- Description
- Category
- Relevant characteristics
- Other descriptive text

Keep structured values as metadata where appropriate:

- Price
- Rating
- Review count
- IDs
- Other numerical or categorical filters

The goal is to create one useful text representation for each searchable record.

```
#Making main text for embedding
semantic_df['product_text'] = (
    'Product: ' + semantic_df['product_name'].fillna('') +
    ' | Brand: ' + semantic_df['brand_name'].fillna('') +
    ' | Category: ' + semantic_df['category_path'].fillna('') +
    ' | Highlights: ' + semantic_df['highlights'].fillna('') +
    ' | Ingredients: ' + semantic_df['ingredients'].fillna('') +
    ' | Variation: ' + semantic_df['variation_type'].fillna('') +
    ' ' + semantic_df['variation_value'].fillna('')
)
print(semantic_df['product_text'][20])
```

Before moving on, print a few records and check:

- Are the correct fields together?
- Are missing values causing strange text?
- Does each text record describe only one item?
- Is important searchable information missing?

Step 2: Generate Embeddings

Load an embedding model such as *Sentence Transformers* and encode the searchable text.

For this implementation, *all-MiniLM-L6-v2* was used. It produces 384-dimensional embeddings.

```
#Embedding
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(
    semantic_df['product_text'].tolist(),
    batch_size=32,
    show_progress_bar=True
)
print(type(embeddings))
print(embeddings.shape)
print(embeddings[20])
```

Useful arguments to help make the process manageable:

- batch_size
- show_progress_bar

The progress bar is particularly helpful when embedding a large number of records, since the process can take some time. Once the embeddings have been generated, save them to a file if they will be reused. This avoids having to regenerate every embedding each time the program is run.

Important: *Use the same embedding model for your stored data and future user queries because they need to be mapped to the same vector space to have the desired semantic search results.*

Step 3: Create Your Vector Database

Create a persistent ChromaDB collection and choose the distance metric.

Note: *When using a persistent ChromaDB client, ChromaDB creates a local database directory that includes a* ***chroma.sqlite3*** *file. SQLite is used internally as part of ChromaDB's persistence architecture; you do not need to write SQL or interact with this file directly. Continue working with the database through ChromaDB's API.*

```
print(product_embeddings.shape)
print(metadata_df.shape)
print(semantic_df.shape)
## Putting embeddings in Chromadb
client = ch.PersistentClient(path = "./chroma_db")

client.delete_collection(name = "SephoraProducts")

products = client.create_collection(
    name = "SephoraProducts",
    metadata = {"hnsw:space": "cosine"}
)

print("ChromaDB collection created successfully!")
```

For each record, store:

- ID
- Embedding
- Original searchable text
- Metadata

```
#Adding products to the vector Database

batch_size = 5000 #Largest bath size is 5000

for start in range(0, len(product_ids), batch_size):
    end = min(start + batch_size, len(product_ids))

    products.add(
        ids = product_ids[start:end],
        embeddings = product_embeddings[start:end].tolist(),
        documents = product_texts[start:end],
        metadatas = product_metadata[start:end])

    print(f"Added {start} to {end - 1}")
print("Products in ChromaDB:", products.count())
```

Then verify that the expected number of records was added.

Step 4: Test Semantic Search

Before adding an LLM, make sure the vector search actually works.

Please see the following steps:

1. Write a natural-language query.
2. Encode it using the same embedding model.
3. Pass the query embedding to ChromaDB.
4. Request several nearest results.
5. Inspect the returned products and distances.

```python
# Searching ChromaDB using semantic similarity
# and applying numerical metadata filters when provided
if where_filter is None:
    results = products.query(
        query_embeddings = [query_embedding],
        n_results = number_of_results,
        include = ["documents", "distances", "metadatas"]
    )
else:
    results = products.query(
        query_embeddings = [query_embedding],
        n_results = number_of_results,
        where = where_filter,
        include = ["documents", "distances", "metadatas"]
    )

print()
print(f"QUERY: {query}")
```

Do not test only obvious queries. Try:

- Clear descriptions
- Different wording from your dataset
- Multiple preferences in one query
- Vague or subjective requirements
- Requirements your dataset may not contain

Tip: *A semantically similar result does not necessarily satisfy every condition in the user's request.*

Step 5: Add Structured Filters

Not everything should be solved using embeddings.

For *"fresh perfume under $50 with at least a 4-star rating"* it should be broken up into semantic (“fresh perfume”) and structured (price ≤ 50, rating ≥ 4).

ChromaDB metadata filters can enforce the structured conditions.

```
    filters = []

    if max_price is not None:
        filters.append({"price_usd": {"$lte": max_price}})

    if min_rating is not None:
        filters.append({"rating": {"$gte": min_rating}})

    if min_reviews is not None:
        filters.append({"reviews": {"$gte": min_reviews}})

    if len(filters) == 0:
        where_filter = None
    elif len(filters) == 1:
        where_filter = filters[0]
    else:
        where_filter = {"$and": filters}

    query_args = {
        "query_embeddings": [query_embedding],
        "n_results": number_of_results,
        "include": ["documents", "distances", "metadatas"]
    }

    if where_filter is not None:
        query_args["where"] = where_filter

    return products.query(**query_args)
```

Important: *A database does not automatically know that: cheap = under $50 or highly rated = rating ≥ 4.5. Only create numerical conditions when the threshold is actually known or has been explicitly defined.*

Step 6: Add Query Interpretation With Ollama

For conversational input, an LLM can clean the user's request before it is embedded.

Your first Ollama call should have one job, to extract the descriptive information needed for semantic search without inventing new requirements.

```python
# Structure returned by Ollama
class SearchRequirements(BaseModel):
    semantic_query: str


# Takes the user's natural-language request and extracts
# the descriptive/product preferences
def understand_query(user_query):

    response = chat(
        model="llama3.1:latest",
        messages=[
            {
                "role": "system",
                "content": """
You are a product search assistant.
Extract only the descriptive product preferences stated by the user.

Keep descriptive terms such as style, scent, size, use case,
product type, and qualitative preferences.

Do not add or infer information that the user did not provide.
Do not turn vague terms such as "cheap", "popular", or "highly rated"
into numerical thresholds.

Remove explicit numerical price, rating, and review constraints.

Return only semantic_query.
"""
```

During testing, pay attention to invented assumptions. If the user says *"highly rated,"* for example, the model should not silently create a numerical threshold.

Step 7: Generate the Final Recommendation

After ChromaDB retrieves candidates, a second Ollama call can turn the raw results into a readable response.

Provide the model with:

- Original user query
- Retrieved product text
- Relevant metadata
- Distance, if you want it used internally for ranking

Then give the model rules for what it may and may not infer.

For example:

- Do not invent product characteristics.
- A rating only establishes the rating.
- Review count does not automatically mean popularity.

- Do not make safety claims from semantic similarity.
- Use only information supported by the retrieved records.
- Do not expose embeddings or vector distances to the user.

```
You are a beauty product recommendation assistant.

Select the 1-3 strongest products from the retrieved candidates that best
match the user's request. Use lower semantic distance as one relevance signal.

For each recommendation, explain specifically why it matches by referring
to evidence in the provided product information, such as the description,
highlights, category, ingredients, or variation. If the relevant information
is not obvious from the product name, explain where the match comes from
naturally.

Only make claims supported by the provided product information.
Do not invent product details or characteristics.
Ratings only indicate rating, and review counts do not prove popularity.
Do not make unsupported medical or safety claims.
Do not use an unrelated characteristic as evidence for another requirement.
```

The database can retrieve more candidates than you ultimately display. For example, you might retrieve

10 candidates but ask the LLM to return only the strongest 1–3.

Step 8: Connect the Pipeline

Your final application only needs to connect the stages:

User Request → Interpret Semantic Preferences → Create Query Embedding → Search ChromaDB → Generate Final Recommendations

Structured metadata filters can additionally be applied when explicit numerical constraints are identified and passed to the retrieval function.

Before considering the system finished, check:

- Does semantic search work before the LLM is added?
- Does the system preserve what the user actually requested?
- When numerical filters are used, are exact numerical requirements passed explicitly rather than inferred?
- Does the LLM invent characteristics?
- Are you asking the system to verify information that is not in your dataset?
- Are weak semantic matches being presented as factual matches?
- Can you identify which stage caused an incorrect result?

Other Ways to Build It

The technology stack used in this tutorial is only one way to build a semantic search system. Several alternatives provide similar capabilities, but they differ in setup, programming requirements, flexibility, and the amount of infrastructure needed. The best choice therefore depends on the application and the technical resources available.

Technology	Best suited for	Advantages	Considerations
ChromaDB	Standalone vector retrieval and smaller prototypes	Free, relatively lightweight, simple to set up, built specifically for vector storage and retrieval	Typically requires Python knowledge for implementation; fewer built-in database management features than larger platforms
MongoDB Vector Search	Combining flexible documents, metadata and vector search	Keeps product data, metadata and embeddings within the same database; supports filtering alongside vector search; graphical interface available through Compass	More setup than ChromaDB; requires learning MongoDB's document/query structure; some Atlas capabilities depend on the deployment/service tier
MySQL HeatWave	Adding semantic search to an existing relational system	Combines familiar SQL operations with vector capabilities; useful when structured relational data already exists	Requires SQL knowledge; more infrastructure and configuration than a lightweight local vector database; HeatWave is a broader managed database platform rather than simply a vector store
Neo4j	Applications where relationships between entities are important	Strong for exploring connections between customers, products, brands and other entities; can complement recommendation systems	Requires learning graph concepts and Cypher; adds complexity if relationships are not actually important to the application